Chapter 13
Service-Based Architecture Style

Service-Based Architecture Style

Introduction

- Neal Ford argued that organizations transition more easily from a monolithic architecture to a service-based architecture than to a microservices architecture.
- Microservice architectures have many benefits, and Ford recommended them for *greenfield projects*. But for organizations which already have a monolithic architecture there are many hurdles to overcome in switching to microservices: breaking apart the code, likely breaking apart a monolithic database, adopting DevOps practices, monitoring, dealing with the network, distributed transactions, etc. Switching to a service-based architecture doesn't require nearly as much change
- A service-based architecture provides more delivery speed than a monolith or Service-Oriented Architecture (SOA) by breaking the code apart in the domain-centric way advocated by microservice and Domain-Driven Design (DDD) proponents. SOA advocates breaking the architecture apart into layers rather than by domain.

Service-Based Architecture Style

Topology

- The basic topology of service-based architecture follows a distributed macro layered structure consisting of a separately deployed user interface, separately deployed remote coarse-grained services, and a monolithic database.
- Services within this architecture style are typically coarse-grained “portions of an application” (usually called domain services) that are independent and separately deployed.
- Because the services typically share a single monolithic database, the number of services within an application context generally range between 4 and 12 services, with the average being about 7 services.

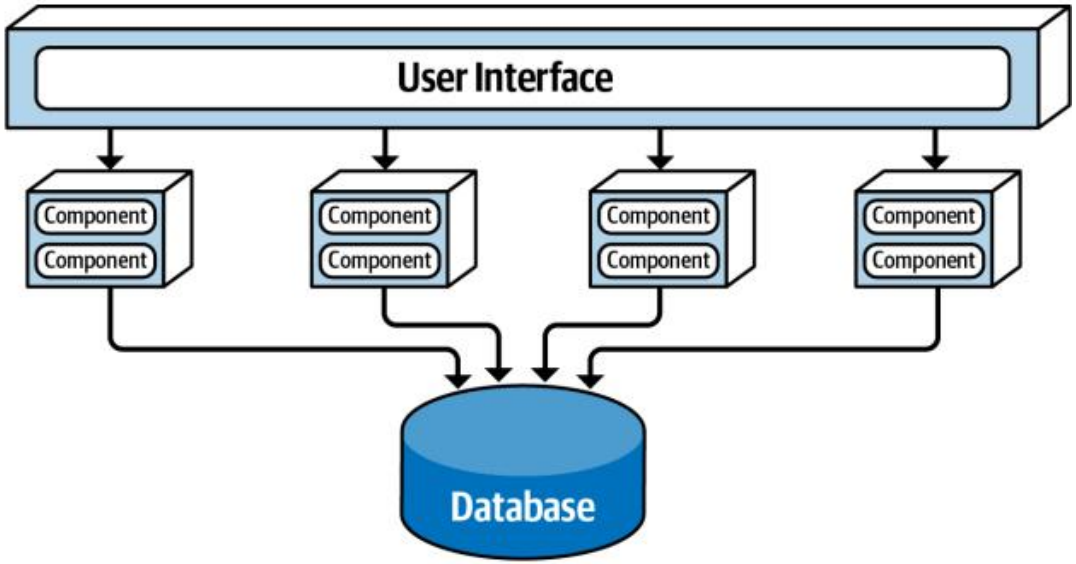


Figure 13-1. Basic topology of the service-based architecture style

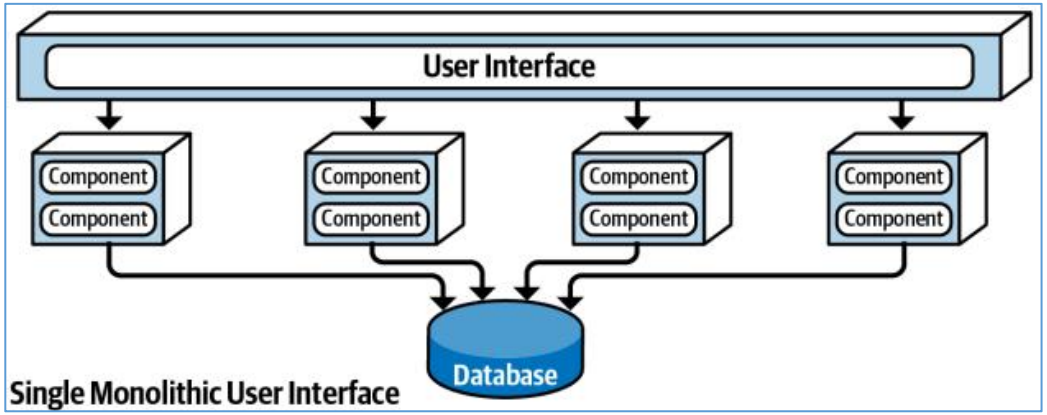
Service-Based Architecture Style

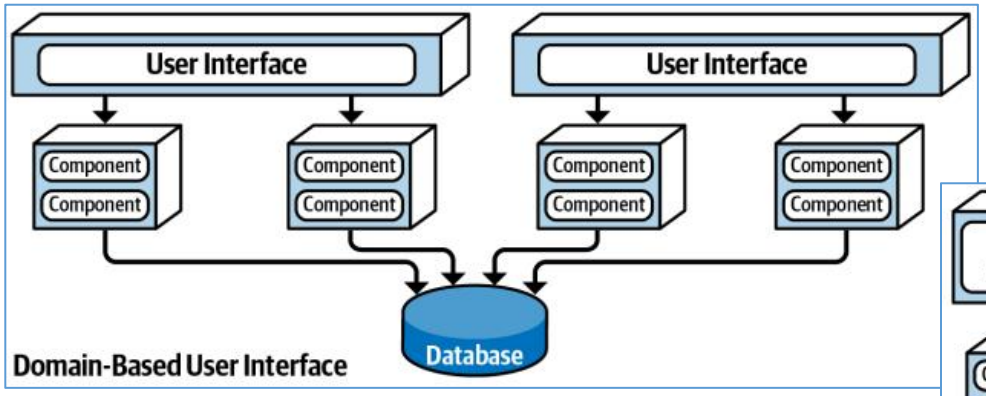
Features

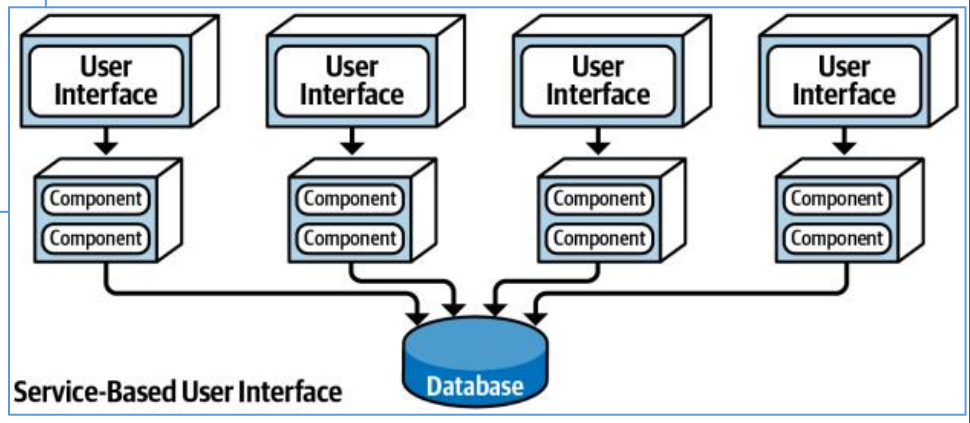
- In most cases there is only a single instance of each domain service within a service-based architecture. However, based on scalability, fault tolerance, and throughput needs, multiple instances of a domain service can certainly exist.
- Services are accessed remotely from a user interface using a remote access protocol.
  - While REST is typically used to access services from the user interface, messaging, remote procedure call (RPC), or even SOAP could be used as well.
  - While an API layer consisting of a proxy or gateway can be used to access services from the user interface (or other external requests), in most cases the user interface accesses the services directly using a service locator pattern embedded within the user interface, API gateway, or proxy.
- One important aspect of service-based architecture is that it typically uses a centrally shared database. This allows services to leverage SQL queries and joins in the same way a traditional monolithic layered architecture would.

Service-Based Architecture Style

User interface variants



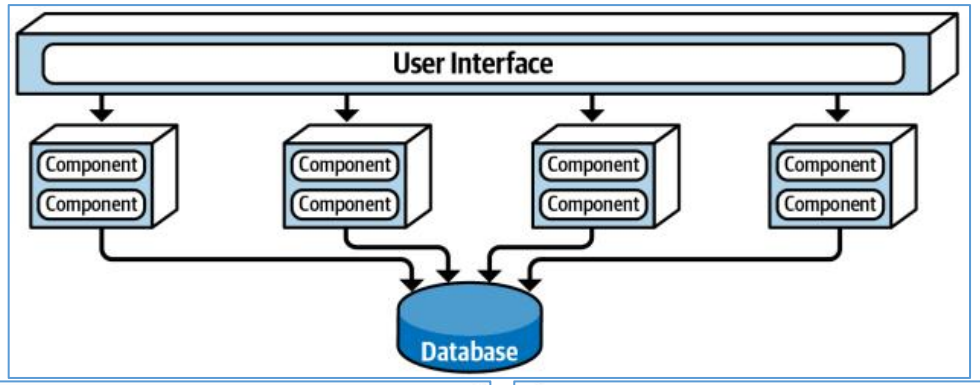


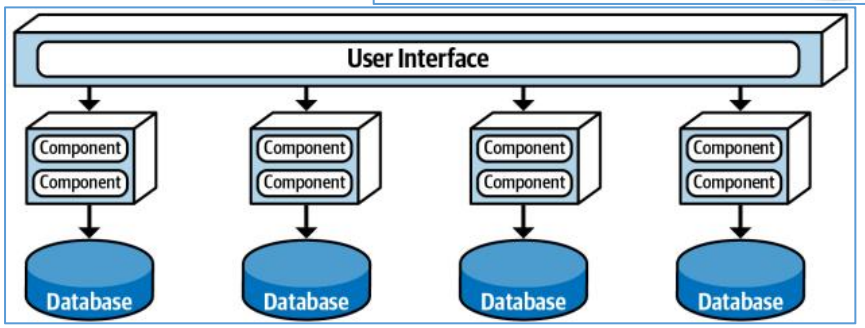


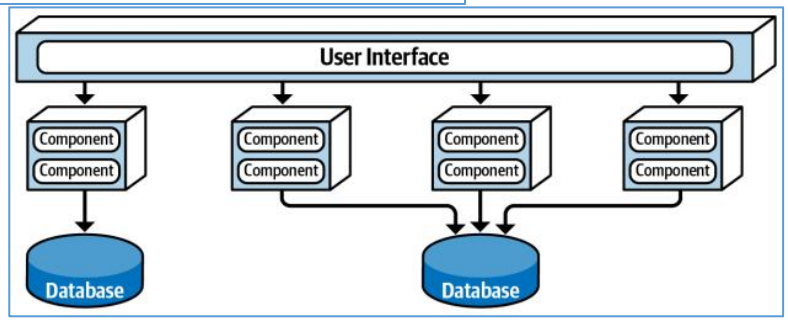
Service-Based Architecture Style

Database variants

- Opportunities may exist to break apart a single monolithic database into separate databases, even going as far as domain-scoped databases matching each domain service (similar to microservices).
- In these cases, it is important to make sure the data in each separate database is not needed by another domain service. This avoids interservice communication between domain services (something to definitely avoid with service-based architecture) and also the duplication of data between databases







Service-Based Architecture Style

Adding API

- It is also possible to add an API layer consisting of a reverse proxy or gateway between the user interface and services.
- This is a good practice when exposing domain service functionality to external systems or when consolidating shared cross-cutting concerns and moving them outside of the user interface (such as metrics, security, auditing requirements, and service discovery).

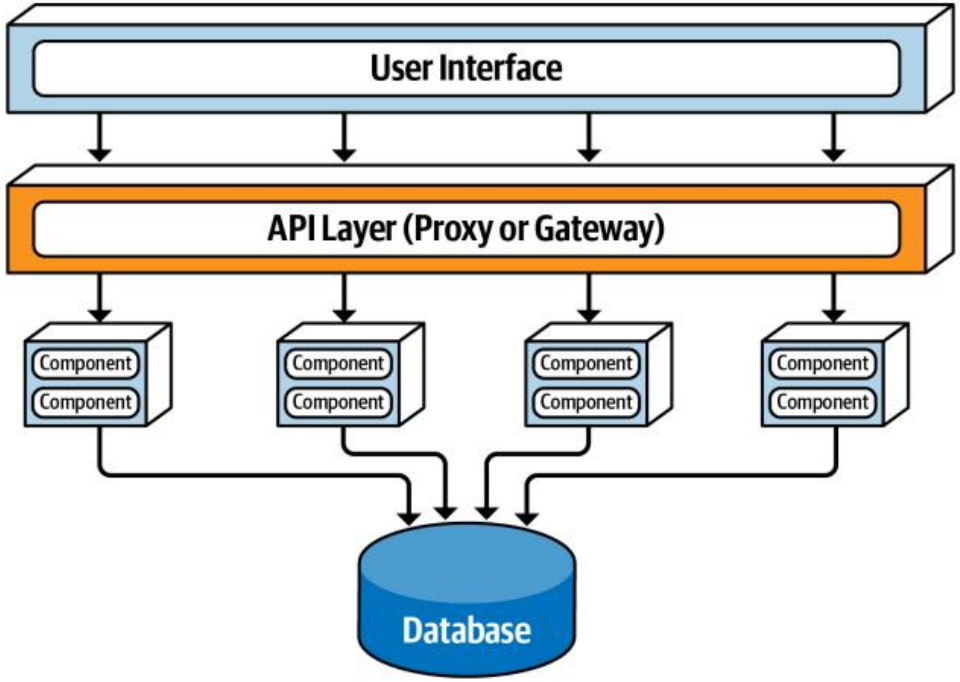


Figure 13-4. Adding an API layer between the user interface and domain services

Service-Based Architecture Style

Service Design and Granularity

- Because domain services in a service-based architecture are generally coarse-grained, each domain service is typically designed using a layered architecture style consisting of an API facade layer, a business layer, and a persistence layer.
- Another popular design approach is to domain partition each domain service using sub-domains similar to the modular monolith architecture style.
- Regardless of the service design, a domain service must contain some sort of API access facade that the user interface interacts with to execute some sort of business functionality. The API access facade typically takes on the responsibility of orchestrating the business request from the user interface.

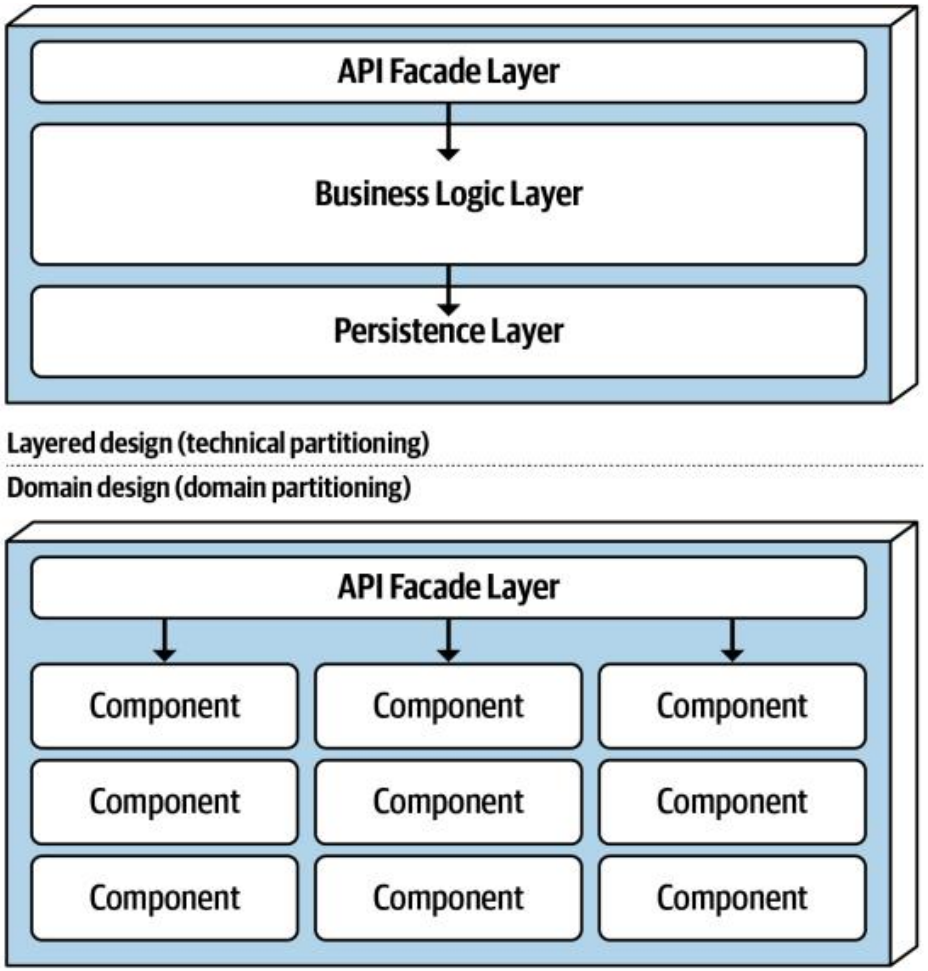


Figure 13-5. Domain service design variants

Service-Based Architecture Style

Database Partitioning

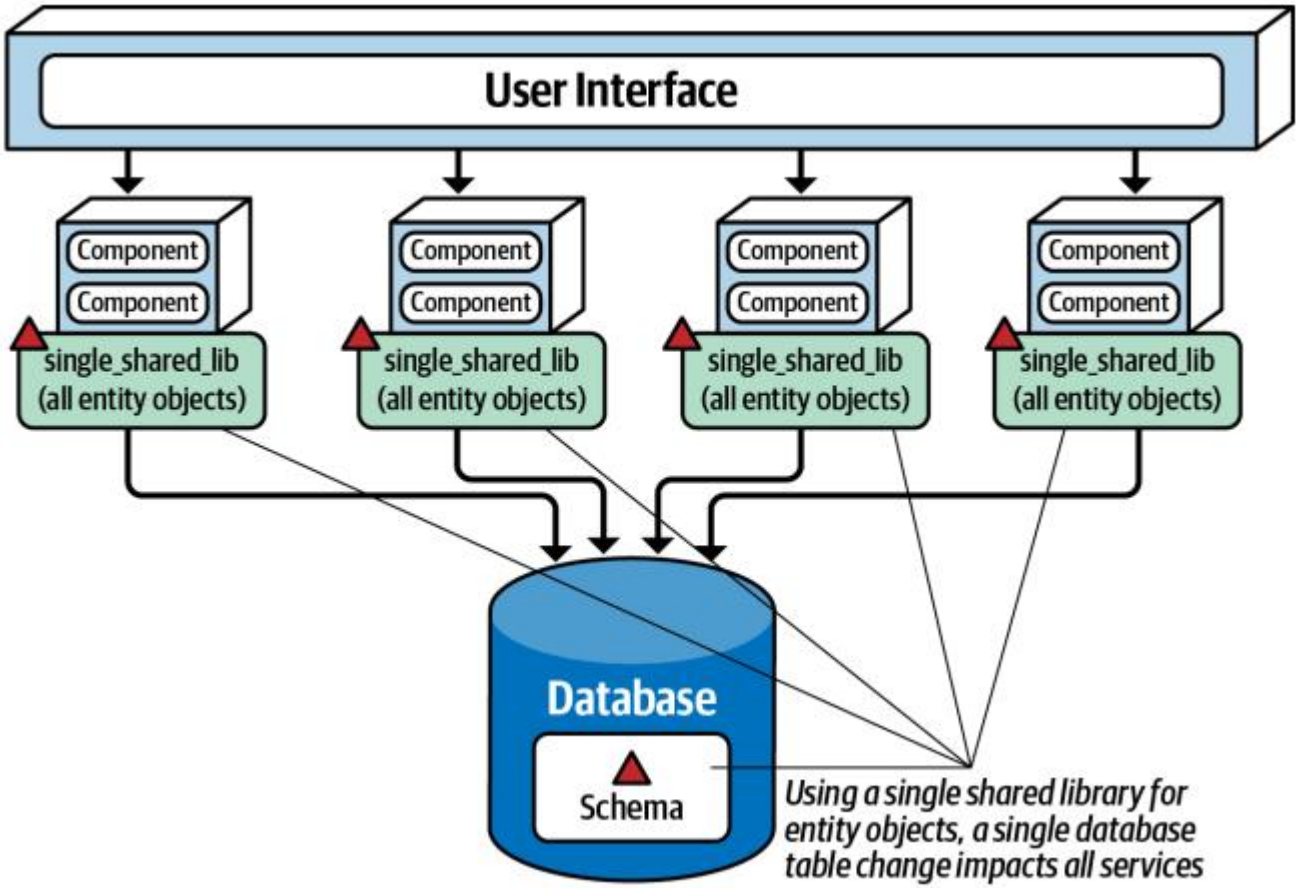


Figure 13-6. Using a single shared library for database entity objects

Service-Based Architecture Style

Database Partitioning

- Make the logical partitioning in the database as fine-grained as possible while still maintaining well-defined data domains to better control database changes within a service-based architecture.

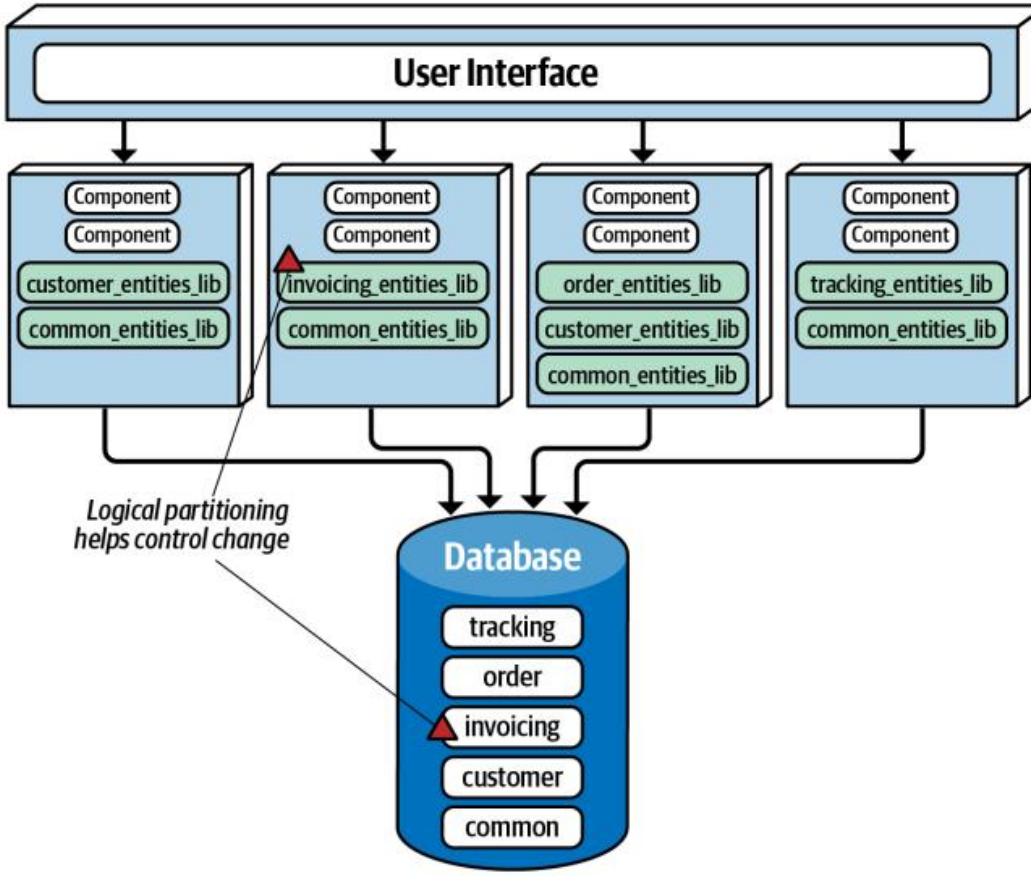


Figure 13-7. Using multiple shared libraries for database entity objects

Service-Based Architecture Style

Example Architecture

- To illustrate the flexibility and power of the service-based architecture style, consider the real-world example of an electronic recycling system used to recycle old electronic devices (such as an iPhone or Galaxy cell phone). The processing flow of recycling old electronic devices works as follows: first, the customer asks the company (via a website or kiosk) how much money they can get for the old electronic device (called quoting). If satisfied, the customer will send the electronic device to the recycling company, which in turn will receive the physical device (called receiving). Once received, the recycling company will then assess the device to determine if the device is in good working condition or not (called assessment). If the device is in good working condition, the company will send the customer the money promised for the device (called accounting). Through this process, the customer can go to the website at any time to check on the status of the item (called item status). Based on the assessment, the device is then recycled by either safely destroying it or reselling it (called recycling). Finally, the company periodically runs ad hoc and scheduled financial and operational reports based on recycling activity (called reporting).

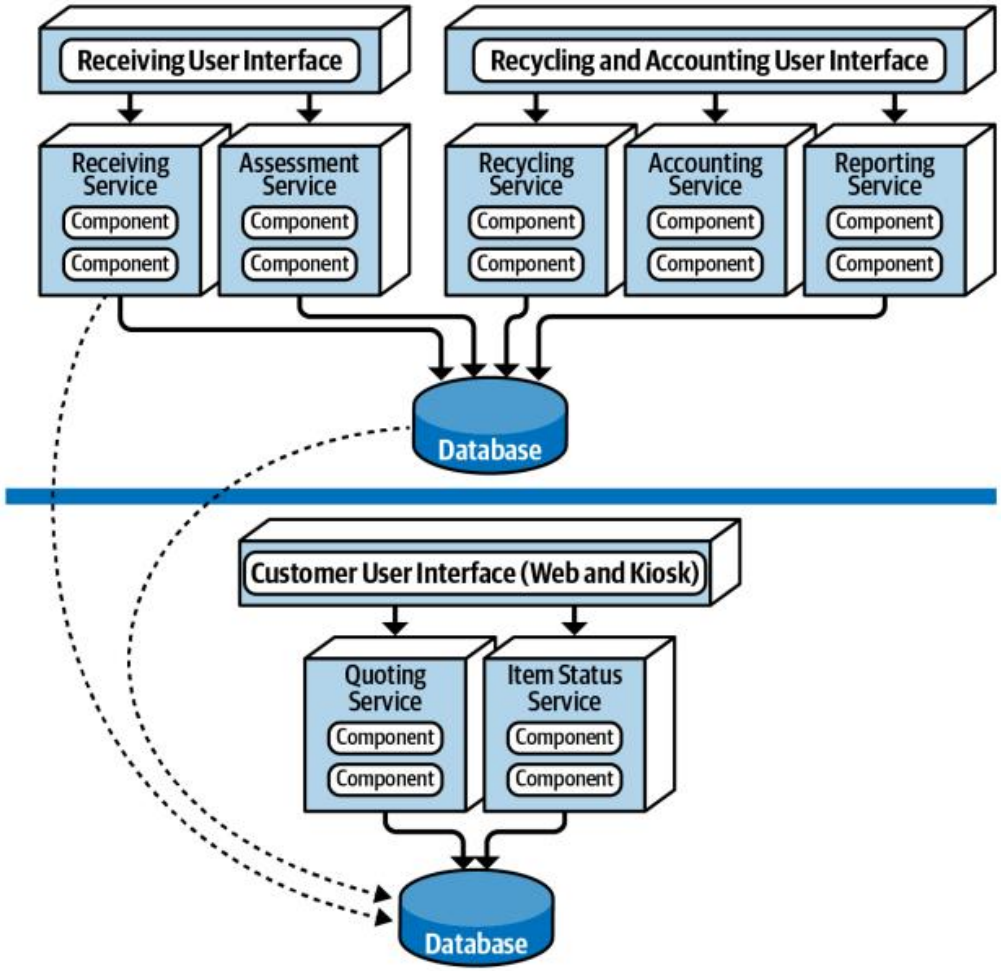


Figure 13-8. Electronics recycling example using service-based architecture

Service-Based Architecture Style

Architecture Characteristics Ratings

- Service-based architecture is a domain-partitioned architecture, meaning that the structure is driven by the domain rather than a technical consideration (such as presentation logic or persistence logic).
- Although service-based architecture doesn't contain any five-star ratings, it nevertheless rates high (four stars) in many important and vital areas.
- Breaking apart an application into separately deployed domain services using this architecture style allows for faster change (agility), better test coverage due to the limited scope of the domain (testability), and the ability for more frequent deployments carrying less risk than a large monolith (deployability).
- These three characteristics lead to better time-to-market, allowing an organization to deliver new features and bug fixes at a relatively high rate.

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★

Service-Based Architecture Style

- https://www.infoq.com/news/2016/10/service-based-architecture/#:~:text=A%20service%2Dbased%20architecture%20provides,layers%20rather%20than%20by%20domain.

- https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/ddd-oriented-microservice
-